UNIVERSITY OF TORONTO
Faculty of Arts and Science

April 2023 Examinations
CSC 384 H1S

Duration: 3 hours
Aids Allowed: a non-programmable calculator

Do **not** turn this page until you have received the signal to start.
In the meantime, write your name and student number and carefully read *all* the information on the rest of this page.

First name (please write as legibly as possible within the boxes)

Last name

Student Number

- Turn off and place all cell phones, smart watches, electronic devices, and unauthorized study materials in your bag under your desk. If it is left in your pocket, it may be an academic offence.
- When you are done your exam, raise your hand for someone to come and collect your exam. Do not collect your bag and jacket before your exam is handed in.
- If you are feeling ill and unable to finish your exam, please bring it to the attention of an Exam Facilitator so it can be recorded before leaving the exam hall.
- This final examination consists of **6 questions** on **30 pages** (including this one), printed on both sides of the paper. When you receive the signal to start, please make sure that your copy of the examination is complete.
- Answer each question directly on the examination paper, in the space provided, and use a “blank” page for rough work. If you need more space for one of your solutions, use one of the “blank” pages and indicate clearly the part of your work that should be marked.
- Remember that, in order to pass the course, you must achieve a grade of at least 40% on this final examination.
- As a student, you help create a fair and inclusive writing environment. If you possess an unauthorized aid during an exam, you may be charged with an academic offence.

Q1 True or False and Short Answers (16 marks)

Q1.1 (2 marks)

Breadth-First Search behaves the same as Uniform-Cost Search if they use the same tie-breaking rule and the costs of all the edges in the search graph are the same.

Circle the best answer: True OR False

Justify your answer in a few sentences:

Q1.2 (2 marks)

Consider Iterative-Deepening Search. If we do not find a solution with depth limit d, IDS increases the depth limit d by 1 and performs Depth-First Search again from scratch. Your friend decides to increase the depth limit by 2 each time. How does this change impact the properties of IDS? Explain in a few sentences.

Q1.3 (2 marks)

Greedy best-first search with a consistent heuristic function is guaranteed to find an optimal solution.

Circle the best answer: True OR False

Justify your answer in a few sentences:

Q1.4 (2 marks)

The variable elimination algorithm alone is sufficient to calculate any filtering probability for a hidden Markov model.

Circle the best answer: True OR False
Justify your answer in a few sentences:

Q1.5 (2 marks)

$p(X) \lor q(1)$ is a valid Skolemization of the formula $\forall X \exists Y. (p(X) \lor q(Y))$ where p and q are predicates, 1 is a constant, and X and Y are variables.

Circle the best answer: True OR False
Justify your answer in a few sentences:

Q1.6 (2 marks)

p and q are predicates, and X and Y are variables. Applying resolution (once or multiple times) on the clauses $(p(X) \lor \neg q(Y))$ and $(\neg p(X) \lor q(Y))$ results in the empty clause.

Circle the best answer: True OR False
Justify your answer in a few sentences:

Q1.7 (4 marks)

X and Y are two variables, and C is a binary constraint between X and Y. Assume that each variable's domain has at least two values.

Assume <**Y**, C(X, Y)> is arc-consistent and <**X**, C(X, Y)> is NOT arc-consistent. To make <**X**, C(X, Y)> arc-consistent, we pruned a value **v** from X's domain, and <**X**, C(X, Y)> became arc-consistent after pruning **v**.

Prove that <**Y**, C(X, Y)> is still arc-consistent after pruning **v** from X's domain.

Q2 Search (18 marks)

1	2	3	4	5
6	7 **Goal**	8	9	10
11	12			15
16		18	19	20
21	22	23 **Start**	24	25

Figure 1 Grid Search

Consider the grid search problem above. The **initial state is 23**. The **goal state is 7**. The three black squares are walls; the agent cannot enter them. The agent can move one square at a time to the left, right, up, or down. The agent can not move diagonally.

We provided the pseudocode for multi-path pruning below.

Generic Search Algorithm **with Pruning**

```
Search (Initial State, Successor Function, Goal Test)
  Frontier = { Initial State }
  Explored = { }
  While Frontier is not empty do
     Select and remove state Curr from Frontier
     If Curr is NOT in Explored
        Add Curr to Explored
        If Curr is a goal state
           return Curr
        Add Curr's Successors to Frontier
  Return no solution
```

Q2.1 Depth-First Search (3 marks)

Execute **Depth-First Search with multi-path pruning on the grid on page 5**. Write down the solution path returned by Depth-First search with multi-path pruning in the table below. Use as many squares in the table as necessary.

Each state has at most four successor states. Add the successor states in **decreasing** order. For example, 23 has three successors: 18, 22, and 24. After removing 23 from the frontier, we will add 23's successors to the frontier in the order of 24, 22, and 18. See step 2 on page 7 for an example.

We provided the pseudocode for multi-path pruning on the previous page.

The solution path returned by depth-first search with multi-path pruning

23									
									7

You can use this page for the rough work on Depth-First Search.
We provided a few steps to get you started.

Depth-First Search with Multi-Path Pruning				
Step	Path to remove	Paths to add	Frontier	Explored set
1		23	23	
2	23	23-24, 23-22, 23-18	23-24, 23-22, 23-18	23
3	23-18			

Q2.2 A* Search (8 marks)

1	2	3	4	5
6	7 **Goal**	8	9	10
11	12			15
16		18	19	20
21	22	23 **Start**	24	25

A Copy of Figure 1 Grid Search

Execute A* Search with multi-path pruning. Complete the table on the next page up to and including step 6. Write each path and its f value in brackets, e.g. 23-18 (4). We recommend ordering the paths in the frontier by their f values first and then by the tie-breaking rule below.

The heuristic function is the Manhattan distance to the goal. For example, the heuristic value for 23 is the Manhattan distance between 23 and 7, the sum of their horizontal distance (1) and their vertical distance (3), which is 4.

The tie-breaking rule: When multiple paths on the frontier have the same f value, choose the path that comes first in **numerical** order. To determine numerical ordering, compare the states in both paths in order. Some examples of numerical ordering are given below.

- 23-18 comes before 25-9 since 23 < 25.
- 23-18 comes before 23-24 since 18 < 24.
- 23-18-25 comes before 23-19-23 since 18 < 19.
- 23 comes before 23-18 since the first states are the same, but 23 is shorter than 23-18.

Q2.2 Answer

A* Search with Multi-Path Pruning				
Step	**Path to remove**	**Paths to add**	**Frontier**	**Explored set**
1		23 (4)	23 (4)	
2	23 (4)	23-18 (4), 23-22 (4), 23-24 (6)	23-18 (4) 23-22 (4), 23-24 (6),	23
3	23-18 (4)			
4				
5				
6				

Q2.3 A* Search Solution (3 marks)

Write down the states in the solution path returned by **A* search with multi-path pruning** in the table below. Use as many squares in the table as necessary.

23									
									7

Q2.4 Optimality of A* Search (2 marks)

For this grid search problem, is A* search with multi-path pruning and the given heuristic guaranteed to find the optimal solution? Circle the best answer and explain.

Your answer: Yes OR No

Your explanations:

Q2.5 UCS versus A* Search (2 marks)

Suppose we execute Uniform-Cost Search with multi-path pruning on this grid search problem. Would UCS with multi-path pruning return the same solution as A* search with multi-path pruning? Circle the best answer and explain.

Your answer: Same solution OR Different solution

Your explanations:

This page is intentionally left blank. You can use this page for rough work.

Q3 Alpha-Beta Pruning (20 marks)

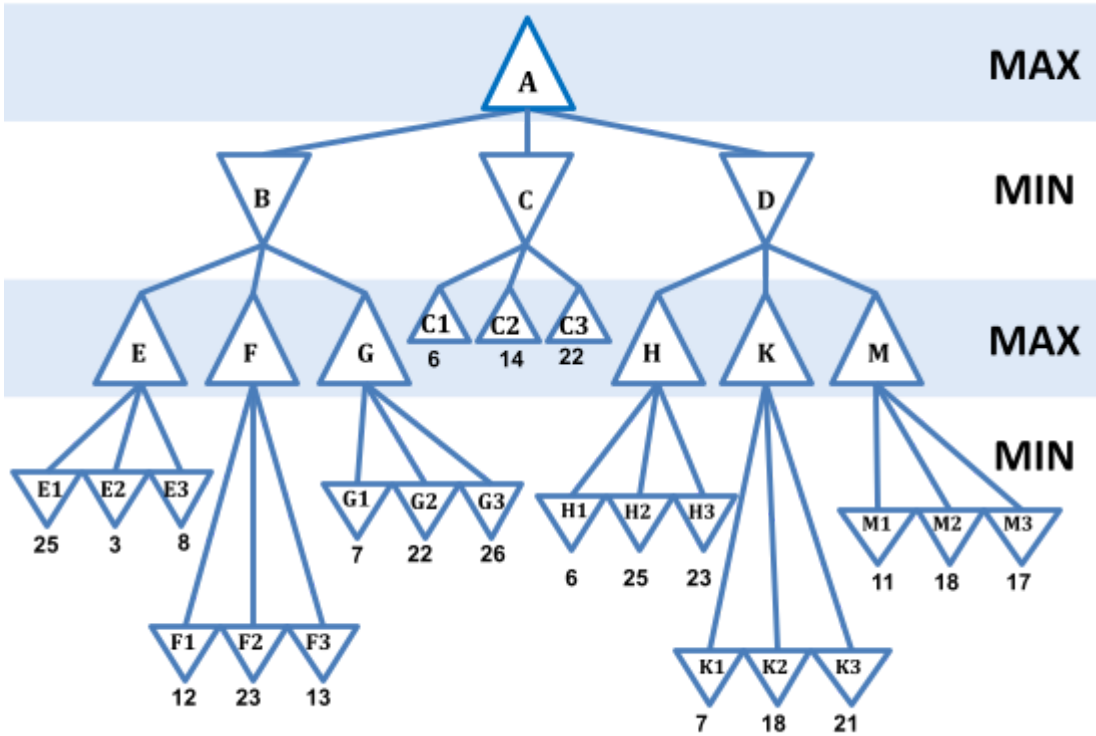


Figure 2 Above

Q3.1 (10 marks)

Execute **Alpha-Beta Pruning** from **left** to **right on Figure 2**.
Circle all the **terminal nodes** that are **pruned (not visited)**.

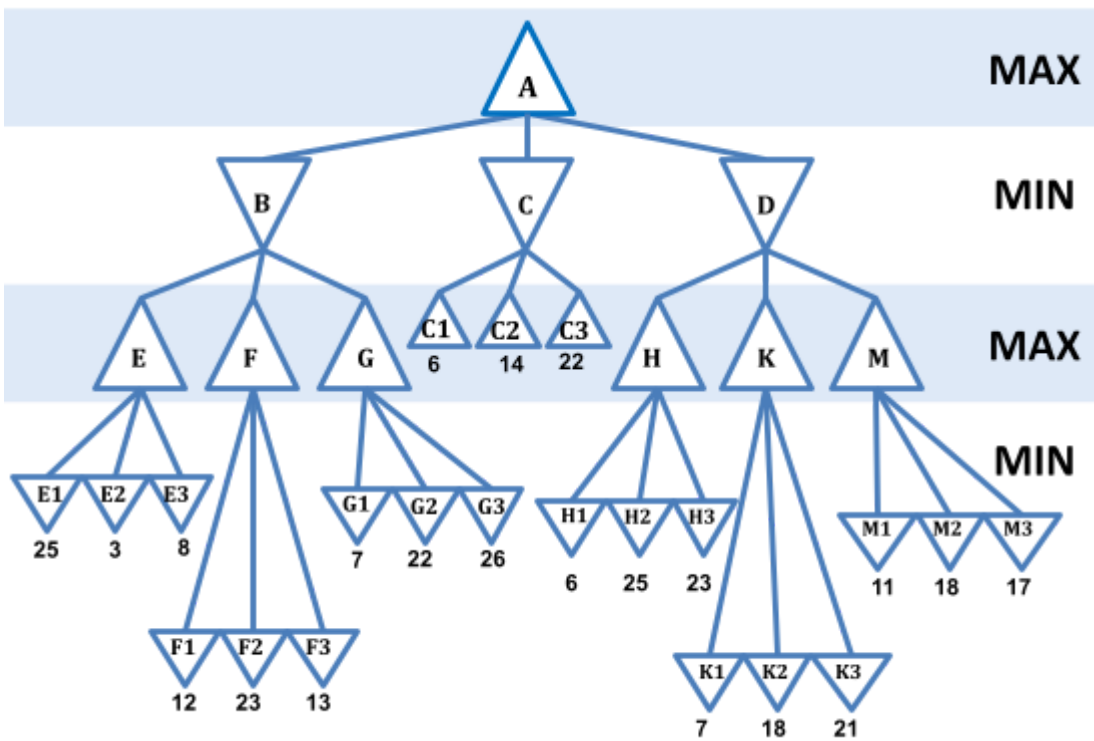
Complete the table below with the **alpha** and **beta** values after visiting each node (after the "return v" line). If a node is not visited, write None. We provided the Alpha-Beta Pruning pseudocode on the next page.

Node	α	β	Node	α	β	Node	α	β
B			C			D		
G			K			M		

See below for the Alpha-Beta Pruning pseudocode.

```
function MAX-VALUE(state, α, β) returns a utility value
    if TERMINAL(state) then return UTILITY(state)
    v ← −∞
    for each action in ACTIONS(state) do
        v ← MAX(v, MIN-VALUE(RESULT(s, action), α, β))
        if v ≥ β then return v
        α ← MAX(α, v)
    return v

function MIN-VALUE(state, α, β) returns a utility value
    if TERMINAL(state) then return UTILITY(state)
    v ← +∞
    for each action in ACTIONS(state) do
        v ← MIN(v, MAX-VALUE(RESULT(s, action), α, β))
        if v ≤ α then return v
        β ← MIN(β, v)
    return v
```



A Copy of Figure 2 Above

Q3.2 (10 marks)

Re-order all the nodes in the game tree to **maximize** the number of terminal nodes pruned by Alpha-Beta Pruning (from left to right). Draw the re-ordered game tree in **Figure 3** below. Label each node with its alpha-numeric name. Label each terminal node with its utility value.

Your **re-ordering cannot change any parent-child relationship**. For example, since K is D's child in Figure 1, K should still be D's child in Figure 2 below.

After drawing the re-ordered game tree, execute **Alpha-Beta Pruning** from **left** to **right in Figure 3**. Circle all the **terminal nodes** that are **pruned (not visited)**.

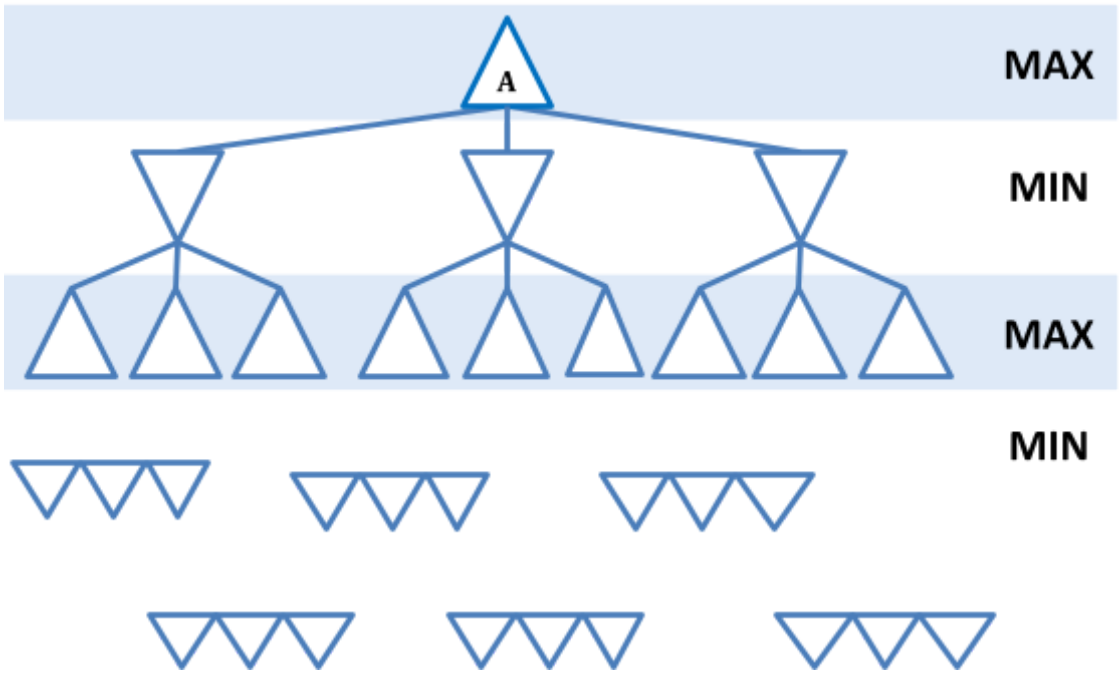


Figure 3 Above

This page is intentionally left blank. You can use this page for rough work.

Q4 The AC-3 Algorithm (12 marks)

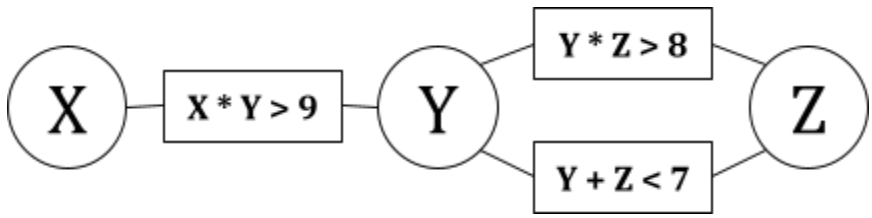


Figure 4 Above

The CSP pictured above has three variables, X, Y, and Z and three binary constraints. We give each constraint a name for convenience.

$$C1\text{: } X * Y > 9,\ C2\text{: } Y * Z > 8,\ C3\text{: } Y + Z < 7$$

Note that there are two binary constraints (C2 and C3) between Y and Z. **Treat these two constraints separately.**

The initial domains of the variables are below.

$$D_X = \{1,\ 2,\ 3,\ 4\},\ D_Y = \{1,\ 2,\ 3,\ 4\},\ \ D_Z = \{1,\ 2,\ 3,\ 4\}$$

Execute the AC-3 algorithm on this CSP until the algorithm terminates.

Follow the directions below.

- Start with the initial queue on the next page.
- At each step, remove the first (leftmost) arc from the queue.
- Always add arc(s) to the queue's end (right side).
- When adding multiple arcs to the queue, add them in alphabetical/increasing order. For example, <Y, C3> should be added before <Z, C2>. <Y, C2> should be added before <Y, C3>.

Complete the table on the next page. For each step, indicate

- The arc removed
- The value(s) deleted from any domains
- The updated domains of the variables
- Any arc(s) added back to the queue

We completed the first step for you as an example.

Initial Queue: <X, C1>, <Y, C1>, <Y, C2>, <Z, C2>, <Y, C3>, <Z, C3>

Step	**Arc removed**	**Value(s) deleted**	**Updated domains**	**Arc(s) added**
1	**<X, C1>**	**Remove 1, 2 from** D_X	$D_X = \{3, 4\},\ D_Y = \{1, 2, 3, 4\}$ $D_Z = \{1, 2, 3, 4\}$	**None**
Updated Queue: **<Y, C1>, <Y, C2>, <Z, C2>, <Y, C3>, <Z, C3>**				
2				
Updated Queue:				
3				
Updated Queue:				
4				
Updated Queue:				
5				
Updated Queue:				
6				
Updated Queue:				
7				
Updated Queue:				
8				

This page is intentionally left blank. You can use this page for rough work.

Q5 Hidden Markov Models (24 marks)

Consider the hidden Markov model below.

- S_t denotes the hidden state at time t. $S_t = true$ means it rained on day t ($S_t = false$ otherwise).
- E_t denotes the observation at time t. $E_t = true$ means the director brought an umbrella on day t and $E_t = false$ otherwise.

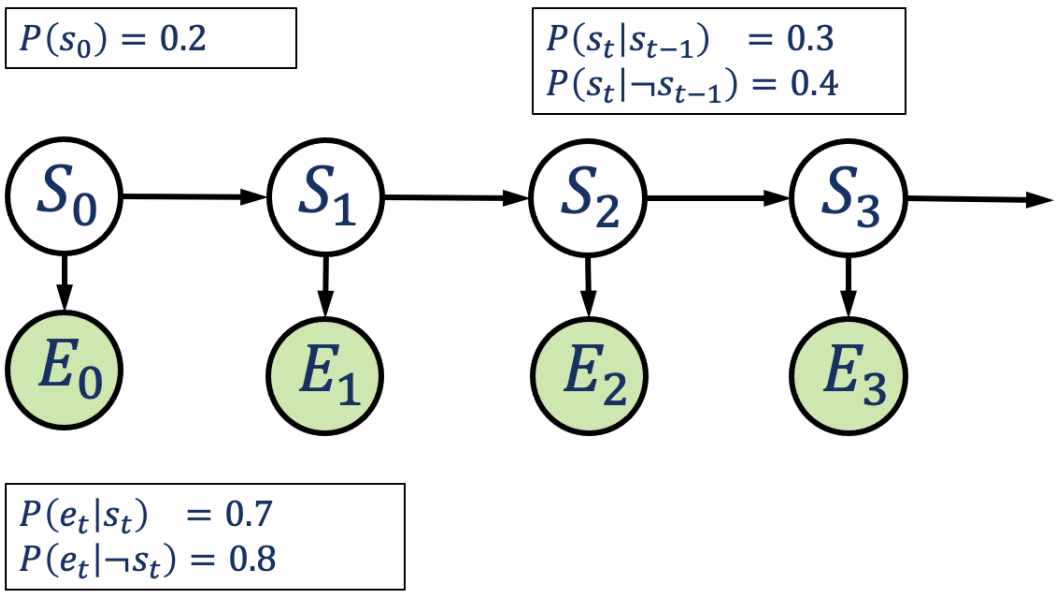


Figure 5 Above

Q5.1 Variable Elimination Algorithm (14 marks)

Consider the HMM model on the previous page.
Calculate $P(s_1 \wedge \neg s_2 | \neg e_0 \wedge e_1)$ using the variable elimination algorithm.

Eliminate the hidden variables in **alphabetical/increasing** order.
For each step, indicate the following.

- Indicate the **operation** (e.g. Restrict, Multiply, Sum out, or Normalize).
- Indicate the **factors** on which you are applying the operation.
- Each operation should **produce a new factor**. Give this factor a unique name and draw a table containing its contents. The table should indicate the variables in the factor and the value for each combination of the variables' values.

We have created some factors for you below.

Feel free to create more factors if needed.

Factor f1

s_0	0.2
$\neg s_0$	0.8

Factor f2

e_0	s_0	0.7
$\neg e_0$	s_0	0.3
e_0	$\neg s_0$	0.8
$\neg e_0$	$\neg s_0$	0.2

Factor f3

s_1	s_0	0.3
$\neg s_1$	s_0	0.7
s_1	$\neg s_0$	0.4
$\neg s_1$	$\neg s_0$	0.6

Your Q5.1 final answer:

$P(s_1 \wedge \neg s_2 \mid \neg e_0 \wedge e_1) =$

Your Q5.1 work starts here.

Your Q5.1 work continues here.

This page is intentionally left blank. You can use this page for rough work.

Q5.2 The Viterbi Algorithm (10 marks)

Consider the HMM model on the previous page.
Assume that **the first three observations are** $\neg e_0$**,** e_1**, and** e_2**.** That is, the director **brought an umbrella on day 2, and didn't bring an umbrella on days 0 and 1.**

You are provided with the information below.

$P(s_0|\neg e_0) = 0.273,\ P(\neg s_0|\neg e_0) = 0.727$

$P(s_0 \wedge s_1|\neg e_0 \wedge e_1) = 0.075,\ P(\neg s_0 \wedge s_1|\neg e_0 \wedge e_1) = 0.267,$

$P(s_0 \wedge \neg s_1|\neg e_0 \wedge e_1) = 0.200,\ P(\neg s_0 \wedge \neg s_1|\neg e_0 \wedge e_1) = 0.458$

Determine the most likely sequence of hidden states given the first three observations by executing the Viterbi algorithm with the information above. Fill in the answer below, and show all your work on the next page.

The most likely sequence of hidden states is:

$\neg s_0$**,** $\neg s_1$**,**

Show all your work for Q5.2 on this page.

This page is intentionally left blank. You can use this page for rough work.

Q6 Knowledge Representation (16 marks)

Q6.1 First-Order Logic (8 marks)

Consider the following predicate symbols.

store(X) - X is a store customer(X) - X is a customer helmet(X) - X is a helmet bike(X) - X is a bicycle	sells(X, Y) - X sells Y buys(X, Y, Z) - X buys Y from Z instock(X, Y) - X is in stock at Y

Convert each sentence below into a first-order logic sentence. Then, convert the first-order logic sentence into clausal form.

Every store that sells bikes also sells helmets.

Convert into a first-order logic sentence.

Convert into clausal form.

Customers cannot buy a product from a store if the store doesn't sell the product.

Convert into a first-order logic sentence.

Convert into clausal form.

Q6.2 Resolution Proof (8 marks)

Consider the following KB. The numbers (1, 2, 3) denote **constants**, and the capital letters (X, Y) denote **variables**. The lowercase letters (p, q, r, s, t) are **predicates,** and f is a **function**.

1. $p(1) \vee \neg q(2) \vee r(3)$	5. $r(Y) \vee s(X) \vee \neg t(Z)$
2. $\neg p(1) \vee q(2) \vee r(3)$	6. $s(X) \vee t(X)$
3. $r(3) \vee s(f(3))$	7. $p(X) \vee r(3)$
4. $r(3) \vee s(1)$	8. $\neg p(X) \vee \neg s(3)$

Give a **refutation proof** that KB entails $r(3)$ by completing the table below. We completed the first step (clause 9) for you as an example.

Clause #	**Rule Applied to Clause #'s**	**Substitutions**	**Derived Clause**
9	**Query Clause**		$\neg r(3)$
10			
11			
12			
13			
14			
15			
16			

This page is intentionally left blank. You can use this page for rough work.